Why LLM Agents Fail: An Empirical Study on Failures in Automated Issue Solving

ANONYMOUS AUTHOR(S)

Automated issue solving seeks to autonomously identify and repair defective code snippets across an entire codebase. SWE-Bench has emerged as the most widely adopted benchmark for evaluating progress in this area. While Large Language Model (LLM)-based agentic tools show great promise, they still fail on a substantial portion of tasks. Moreover, current evaluations primarily report aggregate solving rates, which obscure the underlying causes of success and failure, making it challenging to diagnose model weaknesses or guide targeted improvements. To bridge this gap, we first analyze the performance and efficiency of three state-of-the-art tools, spanning both pipeline-based and agentic architectures, in automated issue solving tasks of SWE-Bench-Verified (a validated subset of SWE-Bench) under varying task characteristics. Furthermore, to move from high-level performance metrics to underlying cause analysis, we conducted a systematic manual analysis of 150 failed instances. From this analysis, we developed a comprehensive taxonomy of failure modes comprising 3 primary phases, 9 main categories, and 25 fine-grained subcategories. Then we systematically analyzed the distribution of the identified failure modes, the results reveal distinct failure fingerprints between the two architectural paradigms, with the majority of agentic failures stemming from flawed reasoning and cognitive deadlocks. Motivated by these insights, we explore failure mitigation through a collaborative Expert-Executor framework, designed to emulate the review processes of human development teams. Experimental results show that our framework successfully resolves 22.2% of previously intractable issues for a state-of-the-art single agent. These findings pave the way for building more robust agents through diagnostic evaluation and collaborative design. [We provide the replication package at TODO.]

CCS Concepts: • **Software and its engineering**; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Automated Issue Solving, Taxonomy of Failure Modes, LLM Agents

ACM Reference Format:
Anonymous Author(s). 2025. Why LLM Agents Fail: An Empirical Study on Failures in Automated Issue Solving. 1, 1 (September 2025), 20 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Automated issue resolution has become a critical challenge in both software engineering and artificial intelligence fields. Its objective is to autonomously identify and repair defective code snippets across entire codebases, guided by reported issues. Developers typically devote the majority of their debugging effort to understanding code and implementing the necessary modifications. Recent advances in large language models (LLMs) and LLM-based agentic tools have significantly advanced the automation of software development tasks. By leveraging their strong capabilities in code understanding and generation, LLMs demonstrate promising performance in tasks such as code generation [2, 3], fault localization [8, 11], program repair [13, 16], and issue solving [12, 17].

SWE-Bench [4] stands out as the most popular benchmark for automated issue solving, comprising 2,294 tasks from 12 well-maintained Python repositories. Each task is paired with an issue description and the corresponding repository snapshot where the issue is to be resolved. The tool

ACM XXXX-XXXX/2025/9-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

to be evaluated is required to generate a patch to address the given issue, and the generated patch is evaluated by running tests associated with the issue. OpenAI further creates SWE-Bench-Verified [6], which contains 500 verified issues from SWE-bench ensuring clear problem specifications and consistent evaluation environments. This benchmark provides a standardized and realistic setting for assessing the strengths and limitations of LLMs in repository-level issue resolution. Its public leaderboard[1] has become a focal point for competition, attracting top solutions from leading AI companies like Anthropic[2] and Cognition AI[3]. This race to the top has steadily driven up reported issue-solving success rates, marking significant progress in automated issue solving.

Despite these advances, LLM agents still fail on a substantial portion of SWE-Bench tasks. For example, OpenHands + CodeAct v2.1 (Claude-3.5 Sonnet) achieved a 53.00% success rate, which represented state-of-the-art open-source solutions as of February 2, 2025. The more critical issue, however, is that current evaluation practices, focused on aggregate pass/fail rates, reveal little about the characterization for these failures. Such success rates effectively treat agents as black boxes, telling us they fail, but not *how* or *when* of the problem-solving process. This lack of diagnostic information makes it difficult for researchers to pinpoint specific weaknesses in agent reasoning or tool interaction, and thus hinders targeted improvements. Without a deeper understanding of the failure modes, further progress risks becoming inefficient. Besides, prior work [] has primarily focused on boosting performance by optimizing prompts, tool integrations, or scaffolding strategies. In contrast, much less attention has been paid to understanding the concrete failures or examining how these failures vary across different architectural paradigms.

To bridge this critical gap, we conduct the first in-depth empirical study of failure modes in state-of-the-art automated issue solving agents. Our study, grounded in the OpenAI-vetted SWE-Bench-Verified benchmark, is framed by four research questions:

RQ1 (Performance and Efficiency Analysis) examines the performance and efficiency of three representative tools (OpenHands [], Tools Calude [], and Agentless [12] in automated issue solving task on SWE-Bench-Verified, where all these tools embody the dominant architectural paradigms and were among the top performers on the SWE-Bench leaderboard at the time of our study). This RQ aims to uncover not only whether these tools succeed but also *how* their performance and efficiency diverge under varying issue characteristics, setting the stage for a deeper diagnosis. Our analysis reveals that, while the tools achieve comparable overall issue-solving rates, they exhibit complementary strengths. Performance consistently declines as task complexity increases, especially for multi-file modification tasks. Moreover, agentic tools frequently become trapped in long, unproductive loops.

To move from high-level performance metrics to underlying cause analysis, **RQ2 (A Taxonomy of Failure Modes)** develops a structured classification of failure modes of the studied tools in RQ1, encompassing 9 primary failure categories and 25 fine-grained subcategories, organized across three issue-solving phases: Location, Repair, and Iteration & Validation. This taxonomy is derived from a detailed manual analysis of 150 issues where at least one studied tool failed.

RQ3 (Characterization of Failure Modes) further examines how these failure modes are distributed across tools and task difficulty levels, uncovers their underlying root causes, and assesses their downstream impact on the issue-solving process. Our analysis reveals that different tools exhibit distinct failure patterns: pipeline tools tend to fail early, often due to incorrect problem localization, whereas agent-based tools more often get stuck in repetitive loops during the later

[1] https://www.swebench.com/index.html
[2] https://www.anthropic.com/
[3] https://cognition.ai/

repair stage. The most common cause of failures is fundamental flaws in the agent's reasoning, which lead it to persist with a flawed plan.

The findings from RQ3 motivates **RQ4 (Mitigation Exploration with a Collaborative Architecture)**, where we propose **Reviewer–Executor** model, a collaborative architecture to address the observed failure patterns. It emulates the review processes of human development teams. The evaluation results on **108** issues previously failed by the baseline `OpenHands` agent show that our collaborative architecture successfully resolves 24 of these challenging cases, achieving performance comparable to top-tier single-agent systems that leverage more powerful proprietary models, such as Claude 4 Sonnet. The improvements are most pronounced in mitigating failures during the **Iterative Verification** stage, such as *bug reproduction failures* and *iteration anomaly failures*. Furthermore, it can systematically prevents strategy defects in the **Repair** phase, such as *evasive repair*. These results provide strong evidence that incorporating an explicit review mechanism substantially improves strategic reasoning and self-correction throughout the automated issue solving lifecycle.

In summary, this paper makes the following contributions:

- **Systematic Empirical Study.** We conduct the first in-depth empirical study contrasting pipeline-based and agent-based tools on SWE-Bench-Verified, revealing their distinct performance characteristics and architecture-specific vulnerabilities.
- **In-depth Failure Analysis and Insights.** We introduce a comprehensive taxonomy for LLM failures in issue resolution. Leveraging this taxonomy, our fine-grained analysis of 150 failure cases provides the first qualitative characterization of why and how different architectures fail, pinpointing root causes like early-stage diagnostic errors in pipelines and unproductive iterative loops in agents.
- **Failure Mitigation Strategy.** Inspired by our empirical findings, we propose a collaborative Expert-Executor framework to address the observed failures, paving the way for more robust and cooperative AI-driven software development.
- **A Publicly Available Dataset of Annotated Failures.** We release our manually annotated dataset of 342 failure instances. To our knowledge, this is the first public resource of its kind, providing a valuable foundation for future research in diagnosing, understanding, and mitigating LLM failures in issue solving tasks.

2 Background & Related Work: Automated Issue Solving

2.1 Benchmark

A critical prerequisite for advancing automated issue resolution is the availability of realistic and rigorous benchmarks that capture the full complexity of real-world software maintenance. The SWE-Bench family of benchmarks has emerged as the de facto standard for evaluating repository-level issue resolution, providing an environment that closely mirrors software engineering workflows.

SWE-Bench [4] comprises 2,294 tasks collected from 12 well-maintained open-source Python repositories, including diverse domains such as scientific computing, web frameworks, and data processing libraries. Each task is defined by three key components: (i) a snapshot of the repository at the time the issue was reported, (ii) the natural language issue description, and (iii) the corresponding ground-truth fix submitted by developers. The goal for an automated framework is to generate a patch that resolves the issue. Evaluation is conducted automatically by running the repository's existing test suite. To be accepted, the model must generate a valid patch in the form of a `git diff`, which can be applied directly to the provided repository snapshot. A solution is considered correct only if the patch applies successfully and all relevant tests pass.

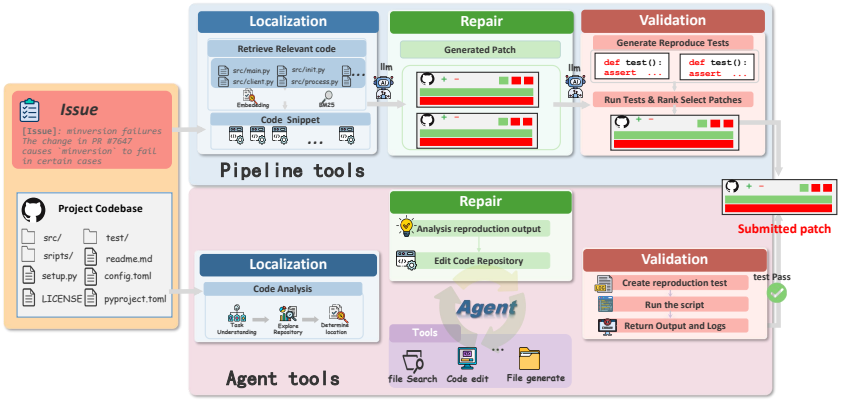


Fig. 1. Workflow of the pipeline-based and agent-based tools.

Although SWE-Bench is highly realistic, its raw form contains noisy and sometimes unsolvable tasks. Common issues include underspecified problem statements, overly rigid or misleading FAIL_TO_PASS tests, and fragile development environments that can cause valid solutions to be rejected. To address this, OpenAI and the original authors released **SWE-Bench-Verified** [6], a curated set of 500 tasks vetted by professional developers. Each task is triple-annotated to ensure clear specifications, valid tests, and consistent execution environments, with problematic samples removed. The benchmark also provides a Docker-based evaluation harness and per-issue difficulty labels that reflect the estimated human time-to-fix (<15 min, 15–60 min, 1–4 h, >4 h). Because the >4 h bucket is rare in our sample (3 cases), we merge 1–4 h and >4 h into *Hard* (≥1 h). Given its reliability and realism, we adopt SWE-Bench-Verified as the foundation for our study, enabling stratified and reproducible analysis.

2.2 LLM-based Issue Solving

LLM-based issue solving frameworks generally follow two distinct approaches: pipeline-based frameworks [12, 14] and agent-based frameworks [], [as shown in Figure 1]. **Pipeline-based frameworks** decompose the task into fixed stages, such as localization, repair, and validation, without dynamic decision-making or agentic behavior.

Aider [] adds a preprocessing step where static code analysis builds a repository map used to constrain retrieval, assemble prompts, and select context windows for the LLM. Agentless [12] bypasses this step, using issue descriptions to retrieve relevant code elements directly. Regarding issue localization, SWE-Fixer [14] uses BM25 retrieval for issue localization, while Agentless combines LLMs with information retrieval methods.

Additionally, Repograph-enhanced Agentless improves code analysis by tracing dependencies and execution flow [7]. Despite these improvements, these frameworks struggle with complex, multi-file issues due to their rigid workflows, limiting flexibility.

In contrast, **agent-based frameworks** operate through iterative observation–action loops, enabling dynamic exploration and adaptive repair. Frameworks such as SWE-Agent [15], OpenHands [10], and Anthropic's Tools+Claude adopt ReAct-style [] reasoning, interleaving natural language thought with tool usage. This design allows agents to iteratively navigate repositories, gather evidence, and refine candidate patches. These methods currently dominate the SWE-Bench-Verified leaderboard, demonstrating strong capability in handling complex issues that pipeline frameworks struggle with. Building on this paradigm, several works have proposed multi-agent extensions that split the repair workflow across specialized roles. For example, AgentScope [1] structures

the process into sequential stages of reproducing, fixing, and testing, each handled by a dedicated agent with its own toolset. MarsCode Agent [5] expands this idea by introducing a larger set of specialized agents and dynamically assembling them into static or adaptive pipelines, supported by a repository-level knowledge graph. While such designs improve modularity and can parallelize certain sub-tasks, they remain fundamentally *workflow-driven*: agents execute statically assigned roles in serial or parallel, passing results along a fixed pipeline. This division of labor alleviates some bottlenecks but does not simulate the proactive, interactive collaboration observed in human development teams, such as critiquing patches or correcting misguided strategies.

3 Methodology

3.1 Research Questions

We formulate four research questions to uncover the current limitations of automated issue resolution frameworks and to inspire future directions for developing more reliable and robust solutions.

- **RQ1: Performance and Efficiency Analysis (§4)**. How do state-of-the-art framework perform in automated issue solving considering success rates, task characteristics, and interaction efficiency?
- **RQ2: A Taxonomy of Failure Modes (§5)**. What are the common failure modes of the studied tools, structured across the key phases of the issue-solving lifecycle?
- **RQ3: Analysis of Failure Modes (§6)**. How are failure modes distributed within this taxonomy, and what architecture-specific vulnerabilities do they reveal, particularly in relation to task difficulties?
- **RQ4: Mitigation Exploration with a Collaborative Architecture (§7)**. To what extent can the identified failures be mitigated using our proposed collaborative Expert–Executor framework?

3.2 Studied Tools and Benchmark

3.2.1 Studied Tools. We select three representative tools that reflect the dominant architectural paradigms in automated issue solving, all of which were top performers on the SWE-Bench leaderboard at the time of our study: ❶ OpenHands-CodeAct-2.1 []: an agentic framework (Rank 3 on SWE-Bench Verified as of February 2, 2025 with Claude-3.5-Sonnet), ❷ Tools + Claude 3.5 Sonnet []: a tool integration framework (Rank 9), and ❸ Agentless-1.5 [12]: the pipeline-based method. To isolate the impact of architectural design on the results, all three frameworks are evaluated under the same backbone LLM (Claude 3.5 Sonnet). This selection allows us to examine how different frameworks architectures affect performance while keeping the backbone model constant.

3.2.2 Benchmark. As mentioned before, we employ SWE-Bench-Verified as the foundation for our study. We utilized publicly available, official experimental results instead of re-running the frameworks ourselves. Specifically, the data for our analysis—including execution logs, generated patches, and pass/fail statuses for all three studied frameworks—was sourced directly from the official **SWE-bench Experiments repository** [9]. The repository hosts official submissions and their execution artifacts in a standardized format.

For our quantitative performance analysis (RQ1), we analyzed the complete official results of the three studied tools on the full 500-issue SWE-Bench-Verified benchmark. This allowed us to establish a baseline understanding of their overall performance and limitations. In our qualitative failure analysis (RQ2 and RQ3), we extract data regarding the solution status across the three selected tools, retaining cases where at least one tool failed to provide a solution, resulting in 329 records. Since manual analysis of all these cases would be both time-consuming, we randomly sampled 150 issues from this set with a 95% confidence level and a margin of error of 5%. This ensures sufficient representativeness while maintaining analytical feasibility. The sample exhibits a

realistic difficulty distribution with 92 medium issues (61.3%), 38 easy issues (25.3%), and 20 hard issues (13.3%), reflecting the complexity spectrum of real-world software engineering tasks.

4 Performance and Efficiency Analysis (RQ1)

4.1 Overall Performance Comparison (RQ1.1)

We evaluated three representative paradigms on 500 issues from SWE-bench Verified: **OpenHands CodeAct v2.1** (interactive agent with optimized scaffolding), **Agentless** (pipeline-style approach), and **Tools Claude** (interactive agent with standard scaffolding). Across the full set, OpenHands solved 53.0% (265/500), Agentless 50.8% (254/500), and Tools Claude 49.0% (245/500). The absolute differences are modest (within 4 percentage points), indicating broadly comparable effectiveness on the SWE-bench Verified benchmark. Notably, OpenHands and Tools Claude share the same base model and a similar toolset, yet still differ by 4.0 points. While we do not attribute this gap to any single factor, practical implementation details (*e.g.*, how file navigation and tool usage are instructed, or workspace path conventions) appear to matter in aggregate.

Beyond overall rates, we also presented the issue-solving distribution diagrams of these tools, as shown in Figure 2. The results suggest complementary strengths across different tools. Specifically, OpenHands and Tools Claude solved 205 issues in common, but each also solved a distinct subset (OpenHands: +60; Tools Claude: +40), and Agentless contributed 30 unique solutions. This indicates that, despite similar aggregate accuracy, the tools excel in solving different instances, suggesting potential benefits for ensemble or portfolio-style usage.

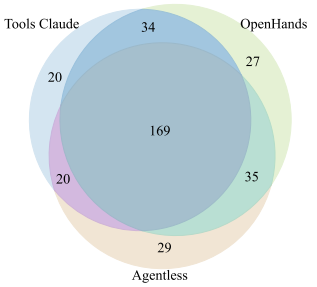


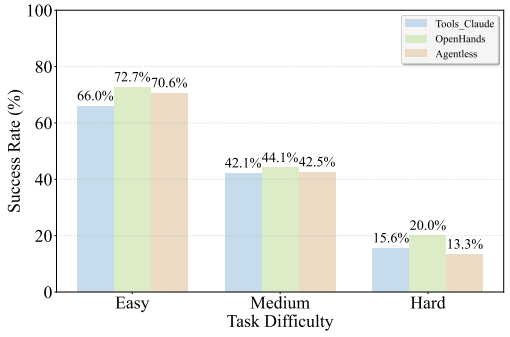
Fig. 2. Venn diagram of resolved cases of the studied tools.

> **Finding 1:** The three studied tools achieve similar overall issue solving rates (49–53%) but exhibit complementary coverage at instance level, suggesting the potential benefits of exploring ensemble methods or selection strategies to optimize issue resolution.

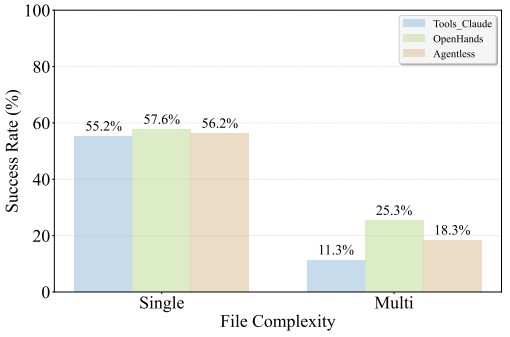
4.2 Task Characteristics and Performance Correlation (RQ1.2)

We further examined how intrinsic issue characteristics correlate with solving rates, focusing on (i) issue description length, (ii) task difficulty (measured by estimated human solving time), and (iii) task complexity, measured by the number of files modified in the ground-truth human patch (single- vs. multi-file).

Issue Description Length. To ensure a robust analysis, we binned the issues into four quartiles based on description token count, creating groups of nearly equal size. The relationship between issue description length and success rate is not consistent, but rather reflects a trade-off between problem complexity and contextual clarity. As shown in Figure 3, the highest performance is achieved on the shortest issues (0-150 tokens), likely because brevity often correlates with simpler, more localized problems that are easier to resolve. However, as descriptions lengthen towards the 500-token mark, success rates decline, suggesting that this range often represents an increase in intrinsic problem complexity without providing compensatory detail. This trend then reverses for the longest issues (>500 tokens), where length is typically a function of rich, prescriptive context—such as detailed error logs, code snippets, and explicit steps—that provides a clear path to a solution, outweighing the task's inherent difficulty.



(a) Issue solving rates across different difficulty levels.



(b) Success rates by file complexity.

Fig. 4. Impact of task characteristics on issue solving performance.

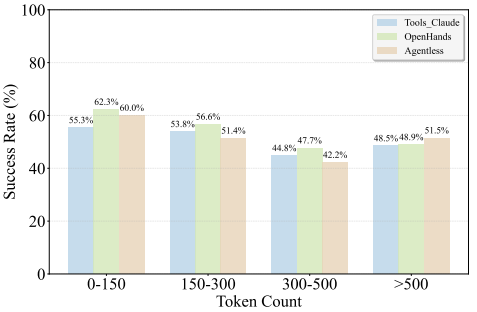


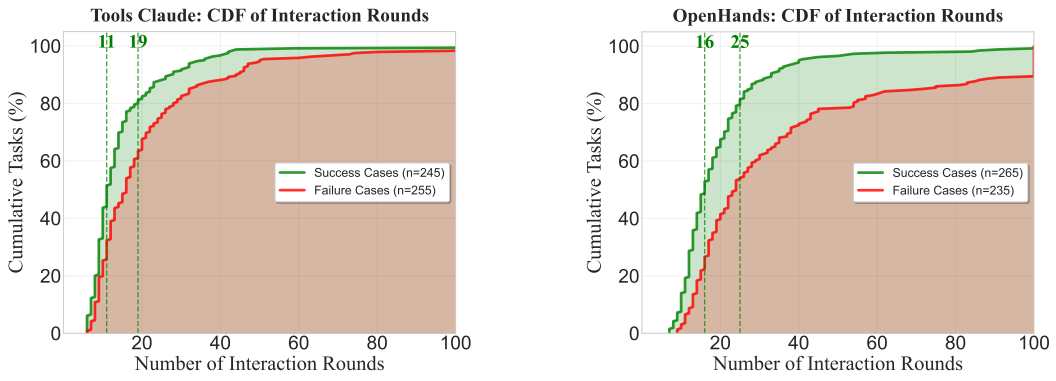
Fig. 3. Impact of issue description length.

Issue Difficulty. As shown in Fig. 4a, the performance of LLM-based agents decreases consistently as the task difficulty increases from Easy to Medium and Hard. For example, OpenHands drops to 44% on Medium tasks, while Agentless and Tools Claude experience declines of more than 20 percentage points. On Hard tasks ($\geq$ 1 h), all frameworks converge to a low success range of 13–20%. Across all three tools, success rates show a strong negative correlation with human-perceived difficulty, indicating that the benchmark's challenges align well with real-world developer effort. These tools excel at solving well-defined problems that a human developer can resolve in under 15 minutes. However, they face significant challenges with more complex, multifaceted problems that require deep reasoning, strategic planning, and sustained problem-solving over extended periods.

Task Complexity. In this section, we investigate how issue complexity, measured by the number of files modified in the ground-truth human patch, affects tool performance. Specifically, "single-file" vs. "multi-file" refers to the number of files modified in the ground-truth human patch for an issue. As shown in Figure 4b, *Single-file* modifications are handled reasonably well by all tools (55-58% success). However, *multi-file* coordination exposes substantial differences: OpenHands maintains a success rate of 25.3%, whereas Tools Claude (Interactive Agent with Standard Scaffolding) drops to 11.3% and Agentless to 18.3%. These sharper declines highlight that cross-file coordination is a key bottleneck.

Agentless, as a pipeline architecture, appears less adept at producing multi-file patches when the human ground truth involves changes across multiple files. Among 497 successful generated Agentless patches, only 6 touched more than one file. For 13 issues whose human fixes required multiple files, Agentless still passed tests with single-file edits in 12 cases. This pattern suggests that the space of valid fixes is not unique and a patch that perturbs fewer locations than the human edit can still make the test suite pass, although such minimal edits may trade off generality or long-term maintainability.

> **Finding 2:** Issue solving rates consistently degrade as task complexity increases, particularly when handling multi-file coordination. OpenHands demonstrates the greatest resilience, maintaining performance even in more complex scenarios. In contrast, Agentless and Tools Claude



(a) Interaction rounds for Tools Claude. (b) Interaction rounds for OpenHands.

Fig. 5. [Efficiency gap between successful and failed tasks.]

experience more significant drops, underscoring the need for architectures specifically designed to handle intricate, multi-step problems and cross-file dependencies.

4.3 Interaction Efficiency (RQ1.3)

To analyze interaction efficiency, we examine the cumulative distribution function (CDF) of interaction rounds for successful and failed tasks in two agent-based tools: OpenHands and Tools Claude (Figure 5a and Figure 5b). As seen from the results, both tools show a clear separation between successful and failed tasks. Tools Claude resolves 80% of its successful tasks within a brisk 19 rounds (median 11), while OpenHands requires 25 rounds for the same success rate (median 16). Crucially, failed tasks exhibit a long-tail effect; OpenHands, for instance, requires 54 rounds to cover 80% of its failures, suggesting it often gets stuck in prolonged, unproductive explorations when a solution is not readily found. The analysis reveals that a majority of successful resolutions occur within approximately 25 interaction rounds, highlighting a critical window for efficient problem-solving.

Finding 3: The value of continued interaction in agent-based problem solving exhibits sharply diminishing returns. Most successes are found within the first 25 rounds. Moreover, agents are far more likely to enter a long-tail failure pattern, engaging in extended but futile explorations. This highlights a critical need for frameworks that can dynamically assess progress and halt unproductive attempts.

5 Failure Modes in LLM-based Automated Issue Solving (RQ2)

[Our performance analysis in RQ1 revealed that even state-of-the-art agents fail on a substantial portion of tasks. More importantly, the efficiency analysis highlighted systemic behavioral patterns, such as agents getting trapped in prolonged, unproductive loops, but it did not explain the underlying causes of these failures. To move beyond *what* happens to *why* it happens, a deeper, qualitative analysis is necessary.]

Answering RQ2 requires a structured classification of error modes tailored to the issue solving lifecycle. To construct such a taxonomy, we first designed and executed a systematic manual analysis procedure to identify and categorize recurrent failure patterns from raw execution traces.

5.1 Procedure for Taxonomy Construction

To construct our taxonomy of failure modes (RQ2) and to characterize failure patterns (RQ3), we designed a **multi-stage manual analysis procedure** that combined open coding, iterative taxonomy refinement, and rigorous annotation protocols. Our process followed established practices in qualitative software engineering research [], while being tailored to the sequential execution traces of LLM-based code agents.

Stage 1: Pilot Analysis and Initial Taxonomy Construction. We first conducted a *pilot analysis* to establish an initial taxonomy of failure modes. For this purpose, we selected 50 execution traces randomly from `OpenHands-CodeAct-2.1`, a representative example of agent tool in our study. Its reasoning traces provided a broad spectrum of failure phenomena, making it suitable for seeding the taxonomy. Two annotators (both Ph.D. students with more than seven years of Python development experience and prior research in automated program repair) independently analyzed sampled failed cases. Following the principles of open coding, each annotator proposed tentative categories by grouping recurrent failure manifestations, based on both framework execution logs and associated test outcomes. After independent analysis, the annotators met to consolidate their findings, merging overlapping categories and clarifying ambiguous definitions.

This stage took approximately 100 person-hours and yielded an *initial codebook* comprising two top-level phases (Diagnosis and Repair) and nine preliminary categories, which served as the foundation for large-scale annotation in Stage 2.

Stage 2: Full Annotation and Taxonomy Refinement. In the second stage, we expanded the annotation to a sample of 150 failed cases drawn evenly across all three studied frameworks (OpenHands-CodeAct-2.1, Tools + Claude 3.5 Sonnet, and Agentless-1.5). Annotation was conducted by **four annotators**, including the two from Stage 1 and two additional researchers (both with over five years of software engineering research and industry programming experience). To facilitate annotation, we pre-processed all execution traces into a structured format and developed a lightweight **web-based annotation platform**. The platform presented annotators with each interaction round of the agent, including its *thought process*, *executed action*, and the corresponding *framework feedback*. Each case was independently annotated by two annotators in parallel, ensuring every instance received at least dual coding. Annotators were instructed to:

(1) Assign one or more failure modes from the codebook.
(2) Identify the specific step(s) in the execution trace where the critical failure emerged.
(3) Document the underlying cause(s) and observable effect(s).

Annotators could flag any instance not covered by the current codebook and submit a label proposal within the platform (additions, deletion or definition edits). Upon submission, the platform automatically notified all annotators; proposals were available for immediate confirmation by any annotator and were reviewed by the four annotators on a rolling basis. Disagreements were resolved via discussion, with final arbitration provided by a senior author with more than ten years of APR research experience. After coding the full set of 150 cases, no additional categories were required beyond the refined Stage-1 codebook, indicating *theoretical saturation* for our sample. Stage 2 required approximately 620 person-hours of effort.

To assess annotation reliability, we computed **Cohen's Kappa** scores across all four annotators. Agreement was measured on two dimensions: (i) assigned failure modes; (ii) the step index in the agent's interaction trajectory where the decisive failure emerged (interactive frameworks only; not applicable to Agentless). Pairwise Kappa values ranged from **0.72 to 0.77** across dimensions; for (ii), **Kappa** was computed on the interactive subset, exceeding the conventional 0.6 threshold for substantial agreement []. Annotation disagreements were first reconciled by the original

annotator pair; the small remainder that persisted after pairwise reconciliation (less than 8% of cases) were resolved in a group discussion with final adjudication by the senior author, yielding a fully adjudicated gold label for every case.

5.2 Taxonomy of Failure Modes

Finally, we identified a total of 342 failure instances across 150 issues. These were organized into three phases—Location, Repair, and Iteration & Validation—encompassing 9 primary failure categories and 25 fine-grained subcategories.

A. Localization Failures. Failures in this initial stage arise when the tool misinterprets the issue or fails to pinpoint the correct code regions requiring modification. Such errors are often catastrophic, as they misdirect the subsequent repair and validation steps, leaving the tool with little chance of success. Figure 6 illustrates a taxonomy of such failure modes.

A1: Issue Misleading. This failure arises when the tool follows misleading implementation suggestions in the issue description rather than independently diagnosing the underlying problem, leading it to modify incorrect locations in the codebase. In the example below[4], the reporter described how transformers were losing pandas DataFrame dtypes. Crucially, the issue description went beyond describing the problem and proposed a detailed, but ultimately incorrect, solution path. Misled by this guidance, the tool focused entirely on output-wrapping logic in `_set_output.py`. However, the true root cause was earlier in the pipeline, where input data was prematurely cast to NumPy arrays, located in `sklearn/base.py` and `sklearn/feature_selection/_base.py`—modules the agent never investigated. By uncritically following the user's suggested roadmap, the agent ended up addressing symptoms rather than resolving the underlying fault.

Fig. 6. Failure modes of Localization.

```
This would mean one changes the `_SetOutputMixin` to add:
* another argument `dtypes` to `_wrap_in_pandas_container`.
* If not None the outputted dataframe uses `astype` to set the `dtypes`.
```

A2: Superficial Information Matching. This failure arises when the tool lacks a deep semantic understanding of the codebase and instead depends on shallow heuristics for localization. It can be further divided into the following sub-categories:

A2.1: Keywords in Description. The tool may rely on naive keyword searches derived from issue descriptions, which can misdirect it to functionally related but incorrect files. For example, in `pylint-dev__pylint-6386`, the description mentioned a buggy "verbose" short option. The tool performed multiple `grep "verbose"` commands, returning many candidates (documentation, config files, checker logic). It incorrectly modified `options.py` containing option metadata, while the actual bug spanned another three files handling command-line argument processing: `pylint/config/argument.py`, `arguments_manager.py`, and `utils.py`.

A2.2: Referred Code. Issue descriptions often reference specific code snippets as examples, and tools may use the provided example code to identify and modify the corresponding module, even if the true fault lies in a different, interacting module. For instance, in `django__django-14034`, the reporter showed that `MultiValueField` failed to enforce required subfields. The agent focused narrowly on this class and patched its validation logic. However, the actual fix required modifying

[4] `scikit-learn__scikit-learn-25102`

widget attribute handling in `boundfield.py`. The tool never explored this location because its investigation was anchored solely to the class mentioned in the example.

A2.3: Error Stack Trace. Tools may also be misled by stack traces from the issue description or their own reproduction scripts, leading them to attempt localized patches without examining the broader call chain to identify the root cause.

```
File "sympy\simplify\radsimp.py", line 1068, in _split_gcd
    g = a[0]
IndexError: tuple index out of range
```

A clear example of this pattern occurred in `sympy__sympy-12420`. The tool observed a crash culminating in the trace below, and it directly modifies the `radsimp.py` file to handle this `IndexError`. However, the root cause was in an upstream file, `sqrtdenest.py`, which was passing an empty tuple to `split_surds`. A proper fix required adding a precondition check in the calling function, not handling the crash downstream.

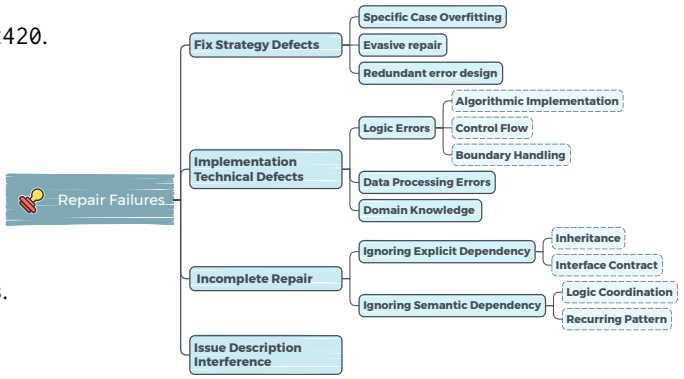


Fig. 7. Failure modes of Repair.

B. Repair Failures. Once a location is identified, the tools generate a code patch to repair the faults and figure 7 illustrates a taxonomy of such failure modes. Failures in this stage can be further divided into the following sub-categories:

B1: Fix Strategy Defects. This category of failure arises from flaws in the tool's high-level repair strategy, preventing the tool from effectively resolving the issue.

B1.1: Specific Case Overfitting. The generated patch may be narrowly tailored to the specific scenario presented in the issue description, leading to brittle and incomplete patches. In the example below, in `django__django-13551`, which reported that password reset tokens were not invalidated when a user's email changed, the tool implemented a solution that directly references the `email` attribute. This solution overfits by assuming the user model always has an attribute named `'email'`. The correct, generalizable solution was to use `user.get_email_field_name()` to dynamically determine the email field.

```
- return str(user.pk) + user.password + str(login_timestamp) + str(timestamp)
+ email = (getattr(user, 'email', '') or '').lower().strip()
+ return str(user.pk) + user.password + email + str(timestamp)
```

B1.2: Evasive Repair. This failure arises when the tool addresses a bug's symptom by wrapping problematic code in defensive constructs like broad `try-except` blocks or null checks, rather than fixing the underlying logical flaw. This prevents crashes but masks root causes.

In `django__django-16938`, a `FieldError` crash occurred during serialization when a Many-ToMany relationship's custom manager with `select_related` conflicted with the serializer's

```
+     qs = getattr(obj, field.name)
+     try:
+         qs = qs.only("pk")
+     except:
+         pass
+     return qs.iterator()
```

`.only("pk")` optimization. The tool's evasive solution uses a bare `except: pass` to prevent crashes, but when conflicts occur, it skips the `.only("pk")` optimization entirely. The correct fix required properly chaining `select_related()` and `only("pk")` calls to resolve the conflict rather than avoid it.

B1.3: Redundant Erroneous Implementation. The tools may introduce overly complex solutions by re-implementing existing logic or ignoring framework extension points, adding redundant code and potentially new bugs.

As shown in the example below (`django__django-13449`), Django incorrectly placed a `CAST` operation inside the `OVER` clause for SQLite window functions. Instead of using the correct adjustment point, the tool manually parsed raw SQL. Django provides the `as_sqlite` method for customizing expression compilation. The correct fix was implementing this method on the `Window` expression class to handle the `DecimalField` case. By ignoring the framework's established design, the tool produced unnecessarily complex and fragile code.

B2: Implementation Details Defects. This occurs when the tool's overall repair strategy is sound, but the implementation contains technical errors, leading to incorrect or failing patches

B2.1: Logic Errors. This group covers fundamental flaws in the patch's reasoning and execution flow. We identify three distinct sub-categories of such failures: *algorithmic error*, *flawed control flow*, and *inadequate boundary handling*. An example of an algorithmic error appears in `sympy__sympy-23413`, where the agent mistakenly conflated two separate mathematical operations. For flawed control flow, in `matplotlib__matplotlib-22871`, the agent oversimplified a conditional statement, failing to preserve the nuanced logic required to handle other cases correctly. Finally, inadequate boundary handling is illustrated by `django__django-11141`, where removing a restrictive check introduced a new bug by causing empty directories—an edge case—to be misclassified.

B2.2: Data Processing Errors. This category refers to mistakes in manipulating or transforming program data. Typical cases include incorrect type casting, variable scope mismanagement (*e.g.*, using a local variable as if it were global), or improper data formatting. Such errors often propagate downstream, resulting in crashes or incorrect behavior.

B2.3: Insufficient Domain Knowledge. This category occurs when the tool fails due to missing knowledge of external frameworks, protocols, or library-specific conventions that are not evident from the local code context. Correct implementation may depend on undocumented conventions or established practices beyond the codebase. As shown in the example below (`django__django-11239`), the goal was to forward SSL parameters from Django's `dbshell` command to the PostgreSQL client `psql`. The tool extracted the parameters but incorrectly attempted to pass them as command-line arguments. This solution failed because `psql` does not accept SSL settings via `-set`; the correct mechanism requires environment variables (e.g., `PGSSLMODE`, `PGSSLCERT`). This defect highlights the tool's insufficient domain knowledge of PostgreSQL's client interface.

B3: Incomplete Repair. This failure occurs when the tool addresses only part of the problem, leaving other necessary changes unimplemented.

```
+    if sslmode:
+        args += ['--set=sslmode=' + sslmode]
+    if sslcert:
+        args += ['--set=sslcert=' + sslcert]
```

B3.1: Ignoring Explicit Dependencies. This happens when the fix may overlook structural dependencies in the code. For example, the tool may modify a child class without updating the parent class, or adjust one implementer of an interface while neglecting others. Such oversights violate explicit structural contracts of the codebase and can introduce inconsistencies or new bugs.

B3.2: Ignoring Semantic Dependencies. The tool may also fail to account for logical dependencies that are not explicitly enforced by the code structure, resulting in partial fixes that leave related parts of the framework broken or out of sync. This includes cases where multiple components must remain consistent or recurring code patterns require simultaneous updates.

B4: Issue Interference. This failure occurs when the tool is misled by incorrect or overly prescriptive suggestions from the issue description instead of independently validating the intended behavior, producing a flawed patch. For example, in `sympy__sympy-15599`, the issue report suggested modifying the internal logic of modular simplification in a particular way. The tool adopted this suggestion directly, which superficially fixed the reported case `Mod(3*i,2)` but introduced inconsistencies for other modular expressions. The correct solution required handling integer coefficients relative to the modulus, which is not captured by the issue's guidance.

C. Iterative Verification Failures. Failure from this stage, particularly in agentic frameworks, involves the loop of testing a patch, interpreting the feedback an agent receives from both its tool interactions and test executions, and refining the solution. Figure 8 illustrates a taxonomy of such failure modes.

C1: Reproduction or Verification Failure. This arises when the tool fails to set up a valid testing environment or correctly interpret its results.

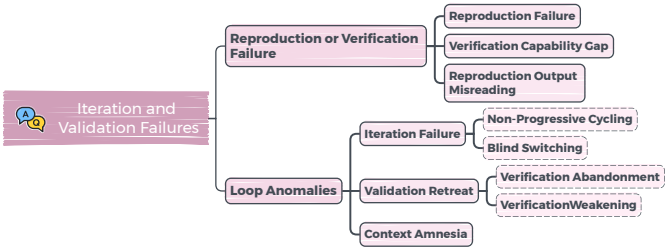


Fig. 8. Failure modes of Iteration.

C1.1: Reproduction/Validation Run Failure. The tool often struggles to reproduce or validate issues because it mishandles project-specific environments and test setups. The tool's reproduction attempts may prune away essential components (*e.g.*, database backends, configuration files), causing tests to fail for setup reasons. For example, in frameworks like Django or Sphinx, failures commonly arise from missing or misconfigured dependencies, incorrect invocation of test runners, or misinterpretation of project-specific scripts and settings.

C1.2: Insufficient Verification Capability. This occurs when the tool's generated test are unable to fully validate the correctness of a fix. For example, in cases involving visual outputs, such as Matplotlib plots or web-based renderings, the agent may confirm only that a function runs without errors or that an image is produced, without verifying whether the output content is actually correct.

C1.3: Reproduction Output Misreading. The tool sometimes misinterprets the feedback from test scripts. For example, even when a test clearly fails or the reproduction script produces output that is unchanged from previous runs—or obviously does not meet the expected criteria—the agent may incorrectly conclude that the test passed. This misreading can lead the agent to prematurely submit a broken or incomplete patch, failing to recognize that the underlying issue remains unresolved.

C2: Iteration Anomalies. This failure often arises when the tool's reasoning within the iterative loop becomes dysfunctional, hindering further progress toward a correct solution.

C2.1: Non-Progressive Iteration. The tool may get trapped in repetitive modification cycles, repeatedly editing the same code fragment or configuration file across multiple interaction rounds without yielding meaningful progress. This often manifests as near-identical patches being regenerated with only trivial or cosmetic differences, such as variable renaming or shifting conditionals, while the underlying defect remains unaddressed.

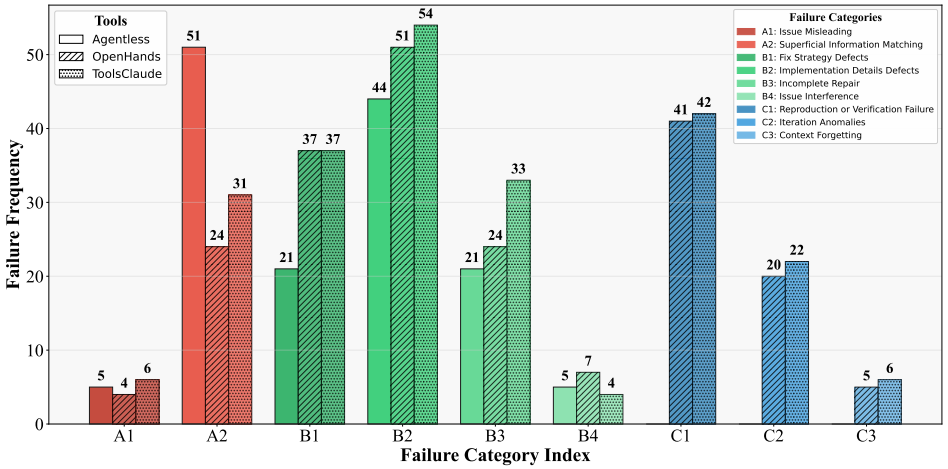


Fig. 9. Distribution of failures across different tools.

C2.2: Blind Strategy Switching. After a failed attempt, the tool may abruptly abandon its previous approach and switch to an unrelated strategy, ignoring lessons from prior failures. This results in fragmented attempts, where consecutive patches target different parts of the codebase or propose incompatible changes. The lack of continuity prevents cumulative refinement, causing the tool to oscillate between disconnected fixes instead of converging on a viable solution.

C2.3: Validation Retreat. To make a test pass, the tool may modify the test case itself—for instance, by commenting out a failing assertion, loosening condition, or adjusting the test setup to accommodate its flawed patch—rather than correcting the underlying code. For example, in `django__django-16454`, after its initial patch failed the provided tests, the tool did not debug its implementation. Instead, it chose to modify the test script, claiming the change would make the test "better match real Django usage". However, this modification simply tailored the test to align with its broken code, rather than ensuring the code met the original test requirements.

C3: Context Amnesia. This failure occurs when the agent loses its original objective or its understanding of the code's current state during a long interaction. For instance, in `django__django-17084`, the agent initially identified the target bug as a `GroupingError` but, after encountering an unrelated setup error, lost its original objective and incorrectly pivoted to fixing this new issue. More frequently, the agent fails to track its own modifications. It may attempt an edit command using a string from the original file content.

6 Analysis of Failure Modes in LLM-based Issue Solving (RQ3)

In this RQ, we go beyond merely cataloging failures to analyze their distribution across tools and task difficulty levels, uncover their underlying root causes, and assess their downstream impact on the issue-solving process.

6.1 Failure Distribution

Figure 9 presents the distribution of primary failure modes across the three studied tools. Overall, the comparison highlights that the distribution of these failures is closely correlated with the tool's architecture. Specifically, each architecture exhibits a distinct "failure fingerprint". The pipeline-based Agentless is defined by its brittleness in the **Understanding & Locating** stage, with over half (**51.3%**) of its failed issues containing a localization error. In contrast, agentic frameworks shift

the failure burden to the **Iteration & Validation** stage, a phase entirely absent in Agentless, where roughly half of their failed issues contain errors (**49.1%** for OpenHands and **51.7%** for ToolsClaude).

We further analyze how failures at different stages (*i.e.*, A: Localization, B: Repair, C. Iterative Verification) are distributed across task difficulty levels, with the results shown in Figure 10. On Easy tasks, a notable portion of failures still occur in the *Localization* stage (41% for OpenHands, 39% for ToolsClaude), suggesting agents can be misled even on simple problems. However, as tasks progress to Medium and Hard, the prevalence of these localization failures generally drops. Concurrently, the proportion of failures in the *Iteration & Validation* stage rises dramatically, increasing from 27% on Easy tasks to 64% on Hard tasks for OpenHands. This pattern indicates that as tasks become more complex, the richer context provided may aid in localization, shifting the challenge from finding the bug to fixing it through a complex iterative process.

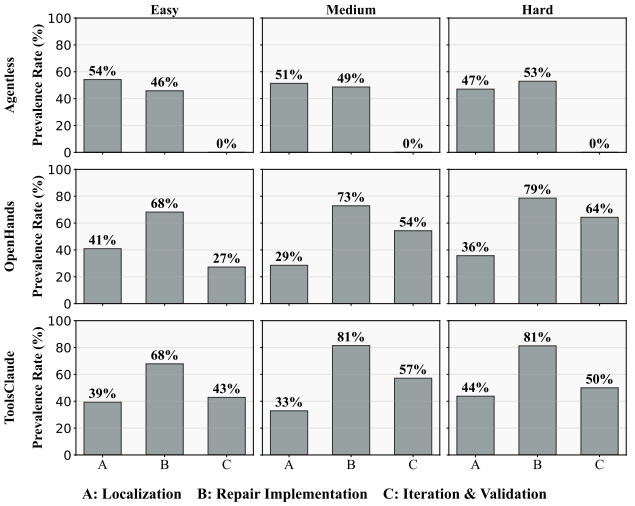


Fig. 10. Failure Stages by Task Complexity.

Besides, the rise in Iteration & Validation failures on more complex problems is driven by the increased prevalence of errors that test an agent's deeper cognitive abilities. For instance, failures in *Algorithmic Implementation* (70.8% in Medium tasks) and *Logic Coordination* (72.3% in Medium tasks) become dominant. Furthermore, on the most convoluted Hard tasks, we observe a notable presence of errors related to long-term strategic coherence, such as *Context Forgetting* and *Blind Strategy Switching*.

> **Finding 4:** Pipeline models mainly fail in localization, whereas agentic models become more prone to Iteration Anomalies failures as tasks grow harder. Yet across all settings, fix implementation remains the dominant bottleneck.

6.2 Causes and Downstream Impacts of Failures

As mentioned in §5.1), during the manual analysis, the annotators were instructed to document both the underlying causes and the observable effects of the identified failures. Building on the distribution patterns identified above, along with the labeled causes and effects, we now analyze why these failures emerge (**causes**) and examine their **downstream impacts**.

6.2.1 Causes. Our manual analysis results suggests that most failures are not simply the result of insufficient context or missing information. Rather, they reflect deeper flaws in the agent's reasoning process. To make sense of these flaws, we distill them into three overarching categories. The most dominant cause (approx. 65% of cases) is **flawed reasoning**, where the tool's internal reasoning logic leads it astray. This is the origin of the most stubborn failures observed in our study. For example, one common manifestation of flawed reasoning is an over-reliance on shallow heuristic—such as fixating on keywords from the issue description—which directly drives the catastrophic *A2: Superficial Matching* failures. More importantly for agentic models, *getting stuck*

in a repetitive loop is the direct cause of *C2.1: Non-progressive Iteration*. This pattern, where an agent becomes fixated on a failing approach, highlights a core vulnerability: the lack of an effective mechanism for self-correction. This inability to pivot from a failing strategy leads to cognitive deadlocks. The second cause is **knowledge deficiency** (approx. 25%), which arises either from missing codebase-specific context (resulting in *B3: Incomplete Repair*) or from outdated domain expertise (*B2.3: Insufficient Domain Knowledge*). Finally, **environmental friction** (approx. 10%) describes the agent's failure to properly interact with its environment, which is the cause of verification issues or tool usage errors.

6.2.2 Downstream Impacts. [The downstream impacts of the failures are significant resource consumption and fruitless explorations.] [In particular, failures caused by **flawed reasoning** typically] drive agents into persistent loop anomalies, where failed issues involve on average **3.5 times** more interaction steps than successful ones. Similarly, [failures derived from] **knowledge deficiency** leads to prolonged but misguided searches through the codebase. [Failures caused by] **environmental friction** also proves costly, especially in complex repositories like `django` or `sphinx`, where agents expend numerous interaction rounds merely attempting to reproduce the issue, often before the core repair task can even begin.

> **Finding 5:** Most failures in agentic tools arise from flawed reasoning. Single agent often gets stuck in cognitive deadlocks, such as persisting with a failed strategy, and lack mechanisms for strategic self-correction. This makes the single-agent paradigm inherently vulnerable, underscoring the need for external oversight.

7 Mitigation Exploration with a Collaborative Architecture (RQ4)

Our findings in RQ3 (§6) reveal that most agent failures arise from **flawed reasoning**, which often leads to cognitive deadlocks. Furthermore, single agent lacks effective self-correction mechanisms to escape these unproductive loops. Building on these insights, we introduce a collaborative framework, the **Expert-Executor Model**, designed to emulate human peer review to mitigate such failures. We implemented this architecture by modifying the source code of the open-source OpenHands framework.

7.1 The Expert-Executor Model

Our framework comprises two distinct but deeply integrated agents:

- **Execution Agent:** An agent responsible for solving issue tasks through code analysis, patch generation, and test execution. Its prompt explicitly requires it to consult the Expert Agent at key decision points—for example, after exploring the repository to validate its understanding of the issue, or before implementing a fix to have its proposed solution reviewed.
- **Expert Agent:** A specialized LLM agent acting as a senior technical lead. Its primary function is to monitor the Execution Agent's workflow, identify potential strategic flaws, and provide corrective guidance. Based on our analysis of failure modes in RQ3 §6, we structured its prompt around our failure taxonomy (developed in §5.2). The prompt provides the name and a detailed explanation for each failure mode, enabling the agent to recognize these specific patterns. [The detailed prompt template can be found in ...]

To solve an issue, the process begins with the **Execution Agent**, which first reads the issue description, analyzes the relevant code, proposes candidate patches, and executes tests to validate them. Unlike single agent tools(like two types analyzed in this article), the **Execution Agent** does not work in isolation. Instead, it collaborates closely with the **Expert Agent**, which serves as a

strategic reviewer to prevent reasoning deadlocks and guide the solution process toward correctness and efficiency. This collaboration is achieved through two primary forms of interaction:

- **Active Consultation:** The Execution Agent is prompted to actively seek guidance at specific times. For instance, before exploring the repository, it shares its understanding of the problem with the Expert Agent. Similarly, before writing the code for a fix, it presents its proposed solution for validation. It is also required to ask for help if it gets stuck.
- **Passive Review:** The Expert Agent periodically reviews the session history to detect flawed patterns like stagnation or strategic errors. If such issues are found, it autonomously intervenes with course-correcting advice to counteract silent failures.

7.2 Experimental Setup and Result Analysis

We implement our model based on **OpenHands**, as it represents a state-of-the-art, open-source agentic framework. To evaluate the effectiveness of our framework, we conduct experiments on the 108 issues from our annotated dataset on which the baseline OpenHands agent had failed. Our goal was to determine if collaborative oversight could resolve these previously intractable cases. The model successfully resolved **24 of these 108** failed issues, achieving a **22.2%** success rate on a challenging set of tasks where a state-of-the-art single agent had already failed. In our Expert–Executor architecture, both the Execution Agent and the Expert Agent use **Claude 3.5 Sonnet**. For comparison, a single-agent baseline (**OpenHands + Claude 4 Sonnet**) resolved only 7 of these cases, underscoring the effectiveness of collaborative oversight.

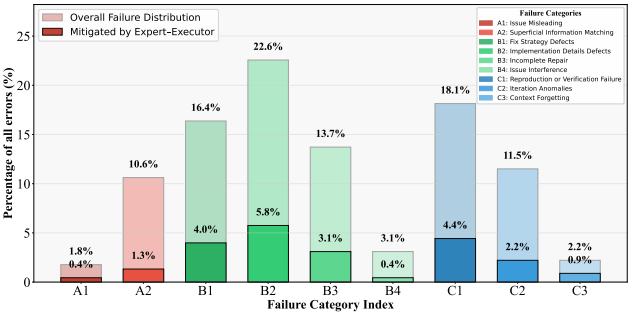


Fig. 11. Distribution of original failure types.

Figure 11 compares the baseline failure distribution of the OpenHands agent with the subset of failures successfully resolved by our Expert–Executor model, highlighting both the dominant error categories and the targeted mitigation achieved through collaborative oversight. The figure shows our Expert–Executor model demonstrates substantial mitigation in the dominant failure categories, particularly (*B1: Fix Strategy Defects*), producing correct code (*B2: Implementation Details Defects*), and accurately interpreting test feedback (*C1: Reproduction or Verification Failure*).

Our analysis of the 24 newly solved instances provides direct evidence of our model's targeted mitigation against these dominant failure types. The Expert's interventions proved most effective against the very patterns that plagued the baseline, successfully correcting the most common iterative error, *Reproduction Output Misreading* (in 6 solved cases), and the most frequent strategic and implementation flaws, including *Evasive Repair* (5 cases) and *Control Flow*

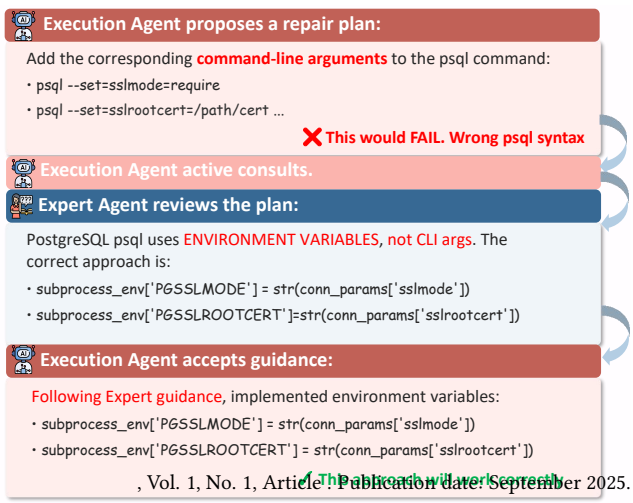


Fig. 12. Example of the Expert-Executor Model.

Defects (5 cases). By succeeding on previously failed tasks, including 16 Medium and 1 Hard-difficulty issues, our Expert-Executor model validates its potential to resolve not just simple bugs, but complex reasoning failures.

[For example, as shown in Figure 12, in `django__django-11239`, when tasked with adding SSL support to Django's `dbshell`, the Executor agent correctly identified the goal but proposed a failed implementation using command-line arguments (e.g., `-set=sslmode=...`) for the PostgreSQL client. However, as required by the prompt, the Executor engaged in **Active Consultation**, presenting its plan to the Expert Agent for validation *before* implementing the fix. The Expert then provided the critical correction, explaining that `psql` requires environment variables (e.g., `PGSSLMODE`) for SSL configuration. This pre-emptive review, initiated by the Executor itself, prevented a predictable failure and guided it directly to the correct.]

> **Finding 6:** Our proposed Expert-Executor model mitigates core reasoning failures of single agents through strategic oversight and peer-review-like collaboration. By breaking cognitive deadlocks and correcting flawed strategies, it turned 22.2% of previously unsolvable problems into solvable ones, showing the effectiveness of an external verification layer in building more robust autonomous agents.

8 Discussion

8.1 Implications

Implications for Researchers. Current benchmarks [] for evaluating issue solving capabilities, dominated by pass/fail rates, obscure critical failure patterns (Finding 4). To foster genuine progress, evaluations should adopt diagnostic benchmarking with failure taxonomies, such as the one we propose, to reveal evaluated tools' specific weaknesses and guide targeted improvements beyond a single success score. Our analysis further highlights cognitive deadlocks as a central failure mode, which scaling monolithic models alone cannot resolve (Finding 5). Addressing this requires a shift toward architectures that explicitly mitigate reasoning flaws—most notably through **collaborative multi-agent frameworks** [] that introduce strategic oversight (Finding 6) and **self-correcting agents** [] capable of recognizing and escaping unproductive loops (Finding 3). Together, these directions point to a research agenda that moves beyond raw scale toward systems designed for diagnosis, collaboration, and adaptive reasoning.

Implications for Practitioners. No single architecture excels at all tasks (Finding 1). Practitioners should adopt a portfolio approach: pipeline-based tools are well-suited for simple, localized bugs, while more resilient agentic frameworks are preferable for complex, multi-file problems (Finding 2). Recognizing each tool's *failure fingerprint* enables more informed choices and better anticipation of likely failure points. At the same time, agents often become trapped in repetitive or unproductive loops (Finding 3). Rather than letting them run indefinitely, practitioners should monitor their progress and **intervene when they are clearly off-track**. Timely, high-level guidance—akin to a senior developer advising a junior—can break deadlocks and save significant time and resources.

8.2 Threats to Validity

Threats to external validity relate to the generalizability of our constructed failure mode taxonomy and findings. Our study is grounded in SWE-Bench-Verified, spanning tasks from 12 diverse Python repositories, and examines three frameworks (OpenHands-CodeAct-2.1, Tools + Claude 3.5

Sonnet, Agentless-1.5) that represent distinct architectural paradigms. This paradigm-level focus suggests that the identified "failure fingerprints" extend beyond specific implementations. Our taxonomy reached *theoretical saturation* during annotation, indicating coverage of the relevant failure space. Still, the analysis is limited to Python and a single LLM (Claude-3.5 Sonnet); future work should test its applicability to other languages and models.

Threats to internal validity relate to the subjectivity of our manual analysis, as different annotators may interpret the same execution trace differently. To mitigate this, we designed a multi-stage protocol: ① a detailed codebook based on a 50-case pilot; ② a custom web platform for standardized annotation; ③ double-coding of each instance by at least two of four annotators; and ④ structured disagreement resolution with senior arbitration. Inter-annotator agreement, measured by Cohen's Kappa (0.72–0.77), indicates substantial reliability, underscoring the robustness of our taxonomy.

9 Conclusion

In this paper, we conduct the first in-depth empirical study on why LLM agents fail at automated issue solving. We introduce a comprehensive failure taxonomy and reveal that different architectures exhibit distinct failure fingerprints: pipeline-based tools are brittle and fail early in Localization, while agentic frameworks succumb to late-stage Iteration loops caused by flawed reasoning and cognitive deadlocks. To mitigate this, we propose a novel collaborative Expert-Executor architecture that mimics human peer review. This model successfully resolved *22.2%* of previously intractable issues by breaking these reasoning deadlocks, demonstrating a clear path toward more robust and reliable AI for software engineering.

References

[1] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, et al. 2024. Agentscope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034* (2024).

[2] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming–The Rise of Code Intelligence. *arXiv preprint arXiv:2401.14196* (2024).

[3] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).

[4] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *ICLR*.

[5] Yizhou Liu, Pengfei Gao, Xinchen Wang, Jie Liu, Yexuan Shi, Zhao Zhang, and Chao Peng. 2024. Marscode agent: Ai-native automated bug fixing. *arXiv preprint arXiv:2409.00899* (2024).

[6] OpenAI. 2024. Introducing SWE-bench Verified. https://openai.com/index/introducing-swe-bench-verified/

[7] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. 2024. Repograph: Enhancing ai software engineering with repository-level code graph. *arXiv preprint arXiv:2410.14684* (2024).

[8] Yihao Qin, Shangwen Wang, Yiling Lou, Jinhao Dong, Kaixin Wang, Xiaoling Li, and Xiaoguang Mao. 2024. Agentfl: Scaling llm-based fault localization to project-level context. *arXiv preprint arXiv:2403.16362* (2024).

[9] SWE-bench Team. [n. d.]. SWE-bench/experiments. https://github.com/SWE-bench/experiments.

[10] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* (2024).

[11] Ratnadira Widyasari, Jia Wei Ang, Truong Giang Nguyen, Neil Sharma, and David Lo. 2024. Demystifying faulty code: Step-by-step reasoning for explainable fault localization. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 568–579.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* (2024).

[13] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 819–831.

[14] Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. Swe-fixer: Training open-source llms for effective and efficient github issue resolution. *arXiv preprint arXiv:2501.05040* (2025).

[15] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[16] Yuwei Zhang, Zhi Jin, Ying Xing, Ge Li, Fang Liu, Jiaxin Zhu, Wensheng Dou, and Jun Wei. 2025. PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. *ACM Transactions on Software Engineering and Methodology* (2025).

[17] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. Autocoderover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1592–1604.